

Annotated Bibliography

Comparing Energy Costs of Parallel DNN Training Techniques

1. Data Parallelism & Distributed Training Frameworks

- [1] Li, S., et al. "PyTorch Distributed: Experiences on Accelerating Data Parallel Training." VLDB, 2020. <https://arxiv.org/abs/2006.15704>

This paper describes the engineering decisions behind PyTorch's DDP implementation, particularly how gradient bucketing and asynchronous all-reduce overlap computation with communication. The authors show that DDP achieves near-linear scaling on standard benchmarks. Key insight for this project: the paper quantifies communication overhead as a fraction of total training time, which is directly relevant to energy cost analysis. The overlap strategy between backward pass and all-reduce is the primary reason DDP outperforms earlier parameter server approaches.

- [2] Sergeev, A., Del Balso, M. "Horovod: Fast and Easy Distributed Deep Learning in TensorFlow." arXiv, 2018. <https://arxiv.org/abs/1802.05799>

Horovod introduced ring-allreduce as a practical replacement for parameter server communication in distributed TensorFlow, achieving 88% scaling efficiency on 128 GPUs. The paper demonstrates that the communication pattern, where each node sends and receives data from neighbors in a ring rather than through a central server; dramatically reduces bottlenecks. This is relevant as a comparison point to PyTorch DDP and AxoNN since all three use different communication strategies for what is fundamentally the same data parallel approach.

- [3] Singh, S., Bhatele, A. "AxoNN: An Asynchronous, Message-Driven Parallel Framework for Extreme-Scale Deep Learning." IPDPS, 2022. <https://arxiv.org/abs/2110.13005>

AxoNN replaces the synchronous NCCL communication used by PyTorch and TensorFlow with asynchronous MPI-based point-to-point messaging, allowing GPU computation to overlap with communication more aggressively. Tested on transformer models with 12-100 billion parameters on up to 384 V100 GPUs, AxoNN achieves 15-25% speedup over Megatron-LM and DeepSpeed. The memory optimization algorithm offloads to CPU periodically, reducing GPU memory by 4x. This is one of three frameworks being directly compared in Algorithm 1 of this project.

- [4] Dean, J., et al. "Large Scale Distributed Deep Networks." NeurIPS, 2012. <https://arxiv.org/abs/1404.5997>

This foundational paper introduced the parameter server architecture for distributed deep learning at Google, where worker nodes compute gradients and send them to centralized parameter servers that aggregate and redistribute updated weights. While largely superseded by all-reduce approaches, the parameter server model established the vocabulary and core tradeoffs, bandwidth vs. computation, that all subsequent distributed training work builds on. Understanding why parameter servers were replaced by ring-allreduce explains why modern frameworks like PyTorch DDP and Horovod perform better at scale.

[5] "Characterizing the Efficiency of Distributed Training: A Power, Performance, and Thermal Perspective." IEEE MICRO, 2025. <https://dl.acm.org/doi/10.1145/3725843.3756111>

This 2025 paper provides a systematic methodology for measuring not just training throughput but power draw, thermal behavior, and energy efficiency of distributed DNN training, which makes it the primary methodological reference for this project. The authors instrument real GPU clusters with power measurement tools and show that different parallelism strategies produce different power profiles even when throughput is similar. Critically for this project, the paper demonstrates that wall-clock speedup and energy efficiency do not always correlate, validating the motivation for measuring both.

[6] Jiang, Y., et al. "Unified Distributed Training of Deep Learning Models via Fine-grained Parallelism." 2020. <https://arxiv.org/abs/1909.02214>

This paper explores how combining data, model, and pipeline parallelism within a single training job can improve GPU utilization compared to pure data parallelism. The authors show that fine-grained parallelism reduces idle GPU time caused by synchronization barriers. Relevant to this project as background for understanding why data parallelism alone leaves performance on the table, and what the next step after baseline measurements would look like.

2. Mixed Precision Training & Quantization

[7] Micikevicius, P., et al. "Mixed Precision Training." ICLR, 2018. <https://arxiv.org/abs/1710.03740>

The original mixed precision training paper from NVIDIA proposes training in FP16 while maintaining FP32 master weights, using loss scaling to prevent gradient underflow. The authors show that for a wide range of models, CNNs, RNNs, and language models, mixed precision matches FP32 accuracy while reducing memory by roughly 2x and increasing throughput on Volta GPUs by 2-3x. This is Algorithm 2's primary reference and establishes the baseline quantization technique against which BF16 and INT8 will be compared in this project.

[8] "Impact of Mixed Precision Techniques on Training and Inference Efficiency of Deep Neural Networks." IEEE, 2023. <https://ieeexplore.ieee.org/document/10146255>

This paper provides an empirical evaluation of mixed precision across multiple GPU generations and model architectures, directly measuring energy consumption alongside speed. The authors find up to 1.9x speedup with mixed precision and show that energy savings scale with the speedup ratio. Uniquely, the paper connects mixed precision to Green AI goals by computing carbon footprint reductions, making it directly relevant to the environmental framing of this project. The methodology for energy estimation used here will inform Algorithm 2's measurement approach.

[9] Fouda, M.E., et al. "Quantized Convolutional Neural Networks: A Hardware Perspective." Frontiers in Electronics, 2025.

<https://www.frontiersin.org/journals/electronics/articles/10.3389/felec.2025.1469802/full>

This 2025 survey covers the full spectrum of quantization techniques for CNNs from a hardware acceleration standpoint, comparing INT8, INT4, binary, and ternary quantization in terms of area, power, and accuracy tradeoffs on different hardware substrates including SRAM-based in-memory computing and neuromorphic chips. Relevant to this project as a guide for selecting which quantization levels to test in Algorithm 2, and for understanding the theoretical energy savings achievable at different precisions.

[10] Rajbhandari, S., et al. "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models." SC'20, 2020. <https://arxiv.org/abs/1910.02054>

ZeRO partitions optimizer states, gradients, and parameters across data-parallel workers rather than replicating them on every GPU, reducing per-GPU memory consumption by up to 8x for optimizer states alone. The paper analyzes three stages of optimization with increasing memory savings and increasing communication overhead. For this project, ZeRO serves as Algorithm 3's primary method, and understanding the memory-communication tradeoff at each ZeRO stage is essential for interpreting energy measurements.

- [11] Rajbhandari, S., et al. "ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning." SC'21, 2021. <https://arxiv.org/abs/2104.07857>

ZeRO-Infinity extends ZeRO by offloading model states to CPU RAM and NVMe storage, enabling training of models with hundreds of billions of parameters on modest GPU clusters. The paper introduces bandwidth-centric partitioning that prioritizes keeping computation on GPU while hiding the latency of CPU/NVMe offloading through prefetching. Directly relevant to Algorithm 3 as the most advanced version of ZeRO, and the offloading behavior introduces new energy consumption patterns, CPU and storage power draw, that are worth measuring.

- [12] "Towards Energy Efficient DNN Accelerator via Sparsified Gradual Knowledge Distillation." IEEE, 2022. <https://ieeexplore.ieee.org/document/9939619/>

This paper proposes jointly combining knowledge distillation with progressive quantization to compress models to ultra-low bit precision while preserving accuracy. The method achieves 2.7x lower energy consumption on compute-in-memory hardware by reducing both the model size and the precision of remaining operations. Relevant as a related technique to Algorithm 2, the combination of quantization and distillation shows that these two approaches are complementary rather than competing, which has implications for how this project frames its Algorithm 3.

- [13] "Distributed Training of Large Language Models: A Survey." ScienceDirect, 2025. <https://www.sciencedirect.com/science/article/pii/S2949719125000500>

This 2025 survey covers the full landscape of distributed LLM training including data parallelism, model parallelism, pipeline parallelism, and memory optimization, with coverage of recent techniques like compute-communication overlap and asynchronous training. Useful as a broad reference that contextualizes all three algorithms in this project within the current state of the field. The survey also discusses energy and resource efficiency as an emerging concern, reinforcing the framing of this project.

3. Memory Optimization & Knowledge Distillation

- [14] Hinton, G., Vinyals, O., Dean, J. "Distilling the Knowledge in a Neural Network." NeurIPS Workshop, 2015. <https://arxiv.org/abs/1503.02531>

The foundational knowledge distillation paper introduces the concept of training a small student model to match the soft probability outputs of a larger teacher model rather than hard labels, arguing that the soft outputs contain rich information about inter-class similarities. This is the starting point for all subsequent distillation work including the energy-efficient variants cited elsewhere in this bibliography. For Algorithm 3, distillation represents a fundamentally different approach to energy efficiency, reducing model size before training rather than optimizing the training process itself.

- [15] "Energy-Aware Deep Learning on Resource-Constrained Hardware." arXiv, 2025. <https://arxiv.org/abs/2505.12523>

This 2025 survey specifically focuses on energy-aware DNN design and deployment on IoT and edge devices, covering quantization, pruning, early exit, and on-device training. The paper is notable for emphasizing energy as a first-class optimization metric rather than a secondary

concern after accuracy, which directly aligns with the framing of this project. The review of energy measurement methodologies, particularly the challenges of accurately measuring dynamic power on GPUs, is useful for designing the measurement approach in Algorithm 1.

[16] "Training Strategies, Computational Consumption, and Memory Control for LLMs." ACM, 2025. <https://dl.acm.org/doi/10.1145/3745533.3745660>

This paper analyzes the memory breakdown of LLM training, showing that optimizer states (particularly Adam's first and second moment estimates) dominate memory usage and that ZeRO stages 1-3 progressively address each component. The paper also shows that mixed precision and ZeRO are complementary, mixed precision reduces activation memory while ZeRO reduces optimizer state memory, suggesting that combining them should yield better results than either alone. This motivates the combination experiment planned for Algorithm 3.

[17] Rajbhandari, S., et al. "ZeRO-Offload: Democratizing Billion-Scale Model Training." USENIX ATC, 2021. <https://arxiv.org/abs/2101.06840>

ZeRO-Offload makes large model training accessible on single-GPU and small-cluster setups by offloading optimizer states and gradients to CPU memory. The paper shows near-linear scaling from 1 to 16 GPUs with minimal CPU overhead when using fast interconnects. Relevant as context for understanding the ZeRO family and for interpreting results when running Algorithm 3 on the course cluster which has limited GPU counts.

4. Pipeline & Model Parallelism (Background)

[18] Huang, Y., et al. "GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism." NeurIPS, 2019. <https://arxiv.org/abs/1811.06965>

GPipe introduces synchronous pipeline parallelism where a model is split across stages and micro-batches are fed through the pipeline to keep all stages busy. The paper shows that pipeline parallelism is complementary to data parallelism and enables training models too large for a single GPU. Relevant to this project as background context, understanding why pipeline parallelism exists helps clarify what data parallelism alone cannot do, and pipeline bubble overhead is one of the energy inefficiencies this project implicitly avoids by focusing on smaller models.

[19] Narayanan, D., et al. "PipeDream: Generalized Pipeline Parallelism for DNN Training." SOSP, 2019. <https://arxiv.org/abs/1806.03377>

PipeDream uses asynchronous pipeline parallelism with weight stashing to avoid the pipeline bubble problem in synchronous approaches like GPipe. The paper shows significant throughput improvements over data parallelism alone on large models. As a course syllabus paper, PipeDream is directly relevant background, it represents one of the parallelism modes that this project is explicitly not implementing, making it useful for the discussion of what this project covers and what it does not.

[20] Shoeybi, M., et al. "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism." arXiv, 2019. <https://arxiv.org/abs/1909.08053>

Megatron-LM implements tensor parallelism within transformer layers by splitting matrix multiplications across GPUs, enabling training of multi-billion parameter models without pipeline parallelism. The paper demonstrates efficient intra-layer parallelism on 512 GPUs. Relevant as a baseline comparison for AxoNN, which was benchmarked against Megatron-LM in the AxoNN paper and shown to achieve better scaling efficiency through asynchronous communication.

5. Environmental Impact of AI Training

[21] "The Carbon and Water Footprints of Data Centers and AI." ScienceDirect, 2025.

<https://www.sciencedirect.com/science/article/pii/S2666389925002788>

This 2025 analysis estimates that AI systems will have a carbon footprint equivalent to New York City and a water footprint comparable to global bottled water consumption in 2025 alone. The paper highlights the lack of transparency in tech company environmental disclosures and calls for mandatory reporting of AI-specific energy and water metrics. This paper provides the primary real-world motivation for this project, if training a single large model consumes this much energy, choosing a more efficient training strategy has measurable environmental consequences.

[22] "Roadmap Shows the Environmental Impact of AI Data Center Boom." Cornell University, 2025.

<https://news.cornell.edu/stories/2025/11/roadmap-shows-environmental-impact-ai-data-center-boom>

Cornell researchers modeled the state-by-state environmental trajectory of AI infrastructure expansion through 2030, finding that current growth rates would produce 24-44 million metric tons of CO2 annually and drain water equivalent to 6-10 million Americans' annual household usage. The study identifies specific geographic and operational choices that could reduce water demand by up to 86%. Relevant for framing this project within a systems-level context, individual training optimization choices aggregate into the infrastructure-level impacts studied here.

[23] Strubell, E., Ganesh, A., McCallum, A. "Energy and Policy Considerations for Deep Learning in NLP." ACL, 2019. <https://arxiv.org/abs/1906.02629>

This influential paper quantified the carbon cost of training large NLP models for the first time, showing that training a single transformer with neural architecture search produces as much CO2 as five cars over their lifetime. The paper sparked the Green AI movement and is cited by virtually every subsequent paper on sustainable deep learning. For this project it provides historical context for why energy measurement in training has become an active research concern, and its methodology for estimating carbon from energy consumption can be adapted.

6. Prasad Lab, UTSA

[24] Mahmud, H. "Converting Autoencoder Based Energy-Efficient and Secure DNN Inference on Edge Devices." PhD Dissertation, UTSA, 2025.

This dissertation from Dr. Prasad's lab at UTSA introduces a Converting Autoencoder (CAE) framework that compresses DNN models by learning to route simpler inputs through smaller sub-networks. The work addresses energy efficiency, inference latency, and adversarial robustness on edge hardware. Chapter 3 contains a detailed methodology for modeling energy and power consumption during DNN inference that is directly applicable to the measurement approach in this project. The dissertation also covers quantization and early-exit techniques, providing lab-specific context for the techniques this project explores.

[25] Singh, S., Bhatele, A. "Exploiting Sparsity in Pruned Neural Networks to Optimize Large Model Training." IPDPS, 2023. <https://arxiv.org/abs/2302.05992>

A follow-up to the AxoNN paper that extends the framework to exploit weight sparsity from pruning during distributed training, reducing both memory and communication. The paper shows that pruned sparse models can be trained more efficiently in parallel without accuracy loss. Relevant as the latest development in the AxoNN ecosystem, understanding how AxoNN has evolved since the 2022 paper helps frame its use in this project and potential future extensions.

[26] "Democratizing AI: Open-source Scalable LLM Training on GPU-based Supercomputers." 2024. <https://arxiv.org/abs/2407.13231>

This paper, from the AxoNN research group, demonstrates fine-tuning a 405-billion parameter LLM on the Frontier supercomputer using AxoNN with a hybrid 4D parallelism strategy combining tensor, pipeline, expert, and data parallelism. The work establishes AxoNN as viable for frontier-scale models, not just benchmarks. For this project, it provides the upper bound context for what AxoNN is designed for, helping explain why its asynchronous communication design offers advantages even at the smaller scale of this project's experiments.

7. Foundational Deep Learning Papers

[27] LeCun, Y., et al. "Backpropagation Applied to Handwritten Zip Code Recognition." Neural Computation, 1989.

The seminal paper demonstrating that backpropagation can train convolutional networks on real-world data, specifically digit recognition from postal zip codes. This paper established the practical viability of gradient-based learning in CNNs and is the direct predecessor to all the training techniques in this project. Presented in class as a foundational reading, it provides the algorithmic basis for understanding why forward and backward pass compute costs differ and why communication overhead in distributed training is a non-trivial fraction of total training time.

[28] He, K., et al. "Deep Residual Learning for Image Recognition." CVPR, 2016.

<https://arxiv.org/abs/1512.03385>

The ResNet paper introduced skip connections that allow gradients to flow through very deep networks without vanishing, enabling training of networks with hundreds of layers. ResNet18, the model used in this project's experiments, is the smallest variant in the family. Understanding the ResNet architecture is necessary for interpreting the memory and compute measurements in this project, since the skip connections create specific memory access patterns that affect GPU cache behavior and therefore power consumption.

[29] Krizhevsky, A., Sutskever, I., Hinton, G. "ImageNet Classification with Deep CNNs."

NeurIPS, 2012. <https://arxiv.org/abs/1404.5997>

AlexNet demonstrated that deep CNNs trained on GPUs could dramatically outperform handcrafted features on large-scale image classification, sparking the modern deep learning era. The paper is also notable for being among the first to report GPU training times and explicitly discuss the computational cost of training. As the catalyst for GPU-based DNN training, AlexNet is the historical starting point for the distributed training challenge that this entire project addresses.

[30] Niu, F., et al. "HOGWILD!: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent." NeurIPS, 2011. <https://arxiv.org/abs/1106.5730>

HOGWILD! proved theoretically and empirically that asynchronous SGD, where workers update shared parameters without locks, converges for sparse problems despite the apparent inconsistency. This work is foundational for understanding asynchronous approaches to distributed training including AxoNN, and explains why synchronization overhead in DDP is not always necessary. The tradeoff between convergence speed and statistical efficiency introduced here is still relevant when comparing synchronous (DDP) vs. asynchronous (AxoNN) approaches in this project.